CPAN226: Network Programming – Advanced Project List (2026 Edition)

Course: CPAN226: Network Programming

Term: Winter/Summer 2026

Format: Individual Project

---

1. Project Overview

The following 12 projects are designed to bridge the gap between foundational socket programming and the modern network landscape of 2026, which prioritizes **Zero Trust Security**, **AI-driven automation**, and **High-Concurrency architectures**. Students may choose one project from the list below.

---

2. Updated Project List (2026)

Python: AI & Automation

1. **AI-Powered Network Traffic Anomaly Detector**: Capture real-time traffic using Scapy. Use a machine learning model to flag suspicious patterns (DDoS or port scanning).
2. **Autonomous Network Resilience Agent**: Develop an "Agentic AI" script that monitors network health and automatically orchestrates a failover to a backup connection during link degradation.
3. **IoT Smart-Gateway with MQTT**: Create a central hub that collects data from multiple simulated IoT devices using the MQTT protocol, providing a real-time dashboard and automated device isolation.

Java: Enterprise & Scalability

4. **High-Concurrency Financial Messaging Server**: Use Java 21 **Virtual Threads** (Project Loom) to build a server capable of handling thousands of simultaneous, low-latency transaction requests.
5. **Microservices-Based Secure File Vault**: Build a distributed system where different services handle Auth, Storage, and Logging, communicating via REST/gRPC secured by **TLS 1.3**.
6. **P2P Academic Resource Sharing Network**: Develop a decentralized peer-to-peer file-sharing system using Java Sockets, focusing on data integrity and handling packet loss.

JavaScript (Node.js): Real-Time & Web

7. **Zero-Trust Real-Time Collaboration Tool**: A workspace using **WebSockets** where every message is verified via **JWT (JSON Web Tokens)** following the "never trust, always verify" security model.
8. **Cloud-Native Network Monitoring Dashboard**: A Node.js application that fetches telemetry from cloud providers (AWS/Azure) via SNMP or APIs to visualize real-time latency and bandwidth.
9. **Offline-First Secure Messenger (PWA)**: A Progressive Web App using **Service Workers** to queue encrypted messages while offline, syncing automatically when connectivity returns.

C: Low-Level & Infrastructure

10. **Custom "Micro" HTTP/1.1 Server**: Build a lightweight web server from scratch using the **Berkeley Socket API**. Manually handle request parsing, headers, and file serving.
11. **Multi-Threaded VPN Gateway**: A C-based proxy that intercepts client traffic and tunnels it through an encrypted channel using **OpenSSL** to a remote endpoint.
12. **Raw Packet Sniffer & Protocol Analyzer**: Use raw sockets to intercept low-level packets (Ethernet/IP). Decode and display headers (MAC addresses, TTL, Port numbers) in the console.

---

3. Submission Requirements

All students must submit the following three components:

1. **Project Report**: A PDF document detailing the architecture, challenges faced, and how network protocols were implemented.
2. **GitHub Repository Link**: A link to a private or public repository containing the full source code and a README.md with setup instructions.
3. **YouTube Video Link**: A link to a private/unlisted video (Max **5 minutes**) demonstrating the functional application and explaining the code logic.

---

4. Evaluation Rubric

Criteria	Description	Weighting

Functional Demo	Successful demonstration of a working application that meets all project requirements.	**50%**
Network Technical Knowledge	Evidence of understanding sockets, protocols (TCP/UDP/MQTT), and data encapsulation.	**30%**
Technical Significance	Alignment with 2026 industry standards and relevance to the CPAN226 curriculum.	**20%**
Video Effectiveness	Clarity of explanation and quality of the 5-minute demonstration.	**10%**
Total		**100%**

5. Important Instructions

- **Individual Effort**: This is an individual project. Plagiarism or the use of AI to generate the entire codebase without understanding will result in a grade of zero.
- **Language Constraints**: Projects must be completed in **Python, Java, JavaScript (Node.js), or C**.
- **Deadline**: Please refer to the course shell for the specific submission date and time.